

New to machine learning? Watch a video course to get practical working knowledge of ML using web technologies [View series \(https://goo.gle/learn-webml\)](https://goo.gle/learn-webml)

Models and layers

In machine learning, a *model* is a function with *learnable parameters* (<https://developers.google.com/machine-learning/glossary/#parameter>) that maps an input to an output. The optimal parameters are obtained by training the model on data. A well-trained model will provide an accurate mapping from the input to the desired output.

In TensorFlow.js there are two ways to create a machine learning model:

1. using the Layers API where you build a model using *layers*.
2. using the Core API with lower-level ops such as `tf.matMul()`, `tf.add()`, etc.

First, we will look at the Layers API, which is a higher-level API for building models. Then, we will show how to build the same model using the Core API.

Creating models with the Layers API

There are two ways to create a model using the Layers API: A *sequential* model, and a *functional* model. The next two sections look at each type more closely.

The sequential model

The most common type of model is the Sequential (<https://js.tensorflow.org/api/0.15.1/#class:Sequential>) model, which is a linear stack of layers. You can create a **Sequential** model by passing a list of layers to the `sequential()` (<https://js.tensorflow.org/api/0.15.1/#sequential>) function:

```
const model = tf.sequential({
  layers: [
    tf.layers.dense({inputShape: [784], units: 32, activation: 'relu'}),
    tf.layers.dense({units: 10, activation: 'softmax'}),
  ]
});
```

Or via the `add()` method:

```
const model = tf.sequential();
model.add(tf.layers.dense({inputShape: [784], units: 32, activation: 'relu'}));
model.add(tf.layers.dense({units: 10, activation: 'softmax'}));
```

IMPORTANT: The first layer in the model needs an `inputShape`. Make sure you exclude the batch size when providing the `inputShape`. For example, if you plan to feed the model tensors of shape `[B, 784]`, where `B` can be any batch size, specify `inputShape` as `[784]` when creating the model.

You can access the layers of the model via `model.layers`, and more specifically `model.inputLayers` and `model.outputLayers`.

The functional model

Another way to create a `LayersModel` is via the `tf.model()` function. The key difference between `tf.model()` and `tf.sequential()` is that `tf.model()` allows you to create an arbitrary graph of layers, as long as they don't have cycles.

Here is a code snippet that defines the same model as above using the `tf.model()` API:

```
// Create an arbitrary graph of layers, by connecting them
// via the apply() method.
const input = tf.input({shape: [784]});
const dense1 = tf.layers.dense({units: 32, activation: 'relu'}).apply(input);
const dense2 = tf.layers.dense({units: 10, activation: 'softmax'}).apply(dense1);
const model = tf.model({inputs: input, outputs: dense2});
```

We call `apply()` on each layer in order to connect it to the output of another layer. The result of `apply()` in this case is a `SymbolicTensor`, which acts like a `Tensor` but without any concrete values.

Note that unlike the sequential model, we create a `SymbolicTensor` via `tf.input()` instead of providing an `inputShape` to the first layer.

`apply()` can also give you a concrete `Tensor`, if you pass a concrete `Tensor` to it:

```
const t = tf.tensor([-2, 1, 0, 5]);
const o = tf.layers.activation({activation: 'relu'}).apply(t);
o.print(); // [0, 1, 0, 5]
```

This can be useful when testing layers in isolation and seeing their output.

Just like in a sequential model, you can access the layers of the model via `model.layers`, and more specifically `model.inputLayers` and `model.outputLayers`.

Validation

Both the sequential model and the functional model are instances of the `LayersModel` class. One of the major benefits of working with a `LayersModel` is validation: it forces you to specify the input shape and will use it later to validate your input. The `LayersModel` also does automatic shape inference as the data flows through the layers. Knowing the shape in advance allows the model to automatically create its parameters, and can tell you if two consecutive layers are not compatible with each other.

Model summary

Call `model.summary()` to print a useful summary of the model, which includes:

- Name and type of all layers in the model.
- Output shape for each layer.
- Number of weight parameters of each layer.
- If the model has general topology (discussed below), the inputs each layer receives
- The total number of trainable and non-trainable parameters of the model.

For the model we defined above, we get the following output on the console:

Layer (type)	Output shape	Param #
dense_Dense1 (Dense)	[null,32]	25120
dense_Dense2 (Dense)	[null,10]	330
Total params: 25450		
Trainable params: 25450		
Non-trainable params: 0		

Note the `null` values in the output shapes of the layers: a reminder that the model expects the input to have a batch size as the outermost dimension, which in this case can be flexible due to the `null` value.

Serialization

One of the major benefits of using a `LayersModel` over the lower-level API is the ability to save and load a model. A `LayersModel` knows about:

- the architecture of the model, allowing you to re-create the model.
- the weights of the model
- the training configuration (loss, optimizer, metrics).
- the state of the optimizer, allowing you to resume training.

To save or load a model is just 1 line of code:

```
const saveResult = await model.save('localStorage://my-model-1');  
const model = await tf.loadLayersModel('localStorage://my-model-1');
```

The example above saves the model to local storage in the browser. See the [model.save\(\)](https://js.tensorflow.org/api/latest/#tf.Model.save) [documentation](https://js.tensorflow.org/api/latest/#tf.Model.save) (<https://js.tensorflow.org/api/latest/#tf.Model.save>) and the [save and load](https://js.tensorflow.org/api/latest/#tf.Model.load) (<https://js.tensorflow.org/api/latest/#tf.Model.load>) guide for how to save to different mediums (e.g. file storage, IndexedDB, trigger a browser download, etc.)

Custom layers

Layers are the building blocks of a model. If your model is doing a custom computation, you can define a custom layer, which interacts well with the rest of the layers. Below we define a custom layer that computes the sum of squares:

```
class SquaredSumLayer extends tf.layers.Layer {  
  constructor() {  
    super({});  
  }  
  // In this case, the output is a scalar.  
  computeOutputShape(inputShape) { return []; }  
  
  // call() is where we do the computation.  
  call(input, kwargs) { return input.square().sum(); }  
  
  // Every layer needs a unique name.  
  getClassName() { return 'SquaredSum'; }  
}
```

To test it, we can call the `apply()` method with a concrete tensor:

```
const t = tf.tensor([-2, 1, 0, 5]);
const o = new SquaredSumLayer().apply(t);
o.print(); // prints 30
```

IMPORTANT: If you add a custom layer, you lose the ability to serialize a model.

Creating models with the Core API

In the beginning of this guide, we mentioned that there are two ways to create a machine learning model in TensorFlow.js.

The general rule of thumb is to always try to use the Layers API first, since it is modeled after the well-adopted Keras API which follows best practices and reduces cognitive load (<https://keras.io/why-use-keras/>). The Layers API also offers various off-the-shelf solutions such as weight initialization, model serialization, monitoring training, portability, and safety checking.

You may want to use the Core API whenever:

- You need maximum flexibility or control.
- You don't need serialization, or can implement your own serialization logic.

Models in the Core API are just functions that take one or more **Tensors** and return a **Tensor**. The same model as above written using the Core API looks like this:

```
// The weights and biases for the two dense layers.
const w1 = tf.variable(tf.randomNormal([784, 32]));
const b1 = tf.variable(tf.randomNormal([32]));
const w2 = tf.variable(tf.randomNormal([32, 10]));
const b2 = tf.variable(tf.randomNormal([10]));

function model(x) {
  return x.matMul(w1).add(b1).relu().matMul(w2).add(b2).softmax();
}
```

Note that in the Core API we are responsible for creating and initializing the weights of the model. Every weight is backed by a **Variable** which signals to TensorFlow.js that these tensors are learnable. You can create a **Variable** using `tf.variable()` (<https://js.tensorflow.org/api/latest/#variable>) and passing in an existing **Tensor**.

In this guide you have familiarized yourself with the different ways to create a model using the Layers and the Core API. Next, see the [training models](/js/guide/train_models) (/js/guide/train_models) guide for how to train a model.

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (https://www.apache.org/licenses/LICENSE-2.0). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2025-05-13 UTC.